

I'd be happy to help. Below is a classroom-ready lecture suitable for **GTU Diploma Engineering (Information Technology)**.

Lecture Content – Unit 1: Introduction to Operating System

Topic: Services of Operating System

Course: Operating Systems (DI03016061)

Programme: Diploma Engineering – Information Technology

Duration: 60 Minutes

1. Hook / Introduction (≈ 5 Minutes)

Classroom Hook

Ask the students:

"Imagine your smartphone suddenly loses its operating system. What would happen if you tried to open WhatsApp, play music, or save a photo?"

Students will quickly realize that none of these tasks would work because the operating system provides the essential **services** required to run applications.

Now ask another question:

"When you click the Save button in MS Word, who actually stores the file on the hard disk? MS Word or the Operating System?"

The correct answer is **the Operating System**. Applications request services, and the OS performs the actual work.

Motivation

Think of the Operating System as the **manager of a hotel**. Guests (applications) ask for services such as rooms, food, electricity, or housekeeping. The manager coordinates all departments and ensures every service is delivered efficiently. Similarly, an Operating System provides services that allow users and application programs to work smoothly.

Learning Outcomes

After this lecture, students will be able to:

- Define Operating System Services.
 - Explain the major services provided by an Operating System.
 - Understand the importance of each service.
 - Relate OS services to everyday computing and industrial applications.
-

2. Core Concepts ($\approx$ 40 Minutes)

What are Operating System Services?

Operating System Services are the facilities provided by the operating system to users and application programs to perform computing tasks efficiently, securely, and conveniently.

Definition

Operating System Services are functions provided by the OS that help users and application software interact with computer hardware efficiently.

Major Services of an Operating System

1. User Interface (UI)

The **User Interface** allows users to communicate with the computer.

Types of UI

- Command Line Interface (CLI)
- Graphical User Interface (GUI)
- Menu-Based Interface
- Touch-Based Interface

Example: Windows Desktop, Android Home Screen, Linux Terminal.

2. Program Execution

The operating system loads a program into memory, executes it, and removes it after completion.

Example: Double-clicking Microsoft Word or opening the Calculator app.

OS Responsibilities

- Load program into RAM
- Allocate CPU time
- Execute instructions
- Terminate the program safely

3. I/O (Input/Output) Operations

Applications cannot communicate directly with hardware. The operating system controls all input and output devices.

Examples

- Keyboard
- Mouse
- Printer
- Scanner
- Monitor
- USB Drive

Real-Life Example: When you print a document, the OS sends print data to the printer driver and manages the print queue.

4. File System Manipulation

The OS organizes, creates, deletes, renames, copies, and protects files and folders.

Examples

- Create a folder
- Copy files
- Delete documents
- Search files

Popular File Systems

- NTFS (Windows)
 - ext4 (Linux)
 - APFS (macOS)
-

5. Communication Service

The OS enables communication between processes running on the same computer or across different computers through a network.

Examples

- Email
- WhatsApp
- File sharing
- Client–Server communication

Fun Fact: Every time you send a message on WhatsApp, the OS manages the network communication before the message reaches the internet.

6. Error Detection

The OS continuously detects and handles errors related to:

- Memory
- CPU
- Storage devices
- Input/Output devices
- Software execution

Example: "Disk Full", "File Not Found", or "Printer Offline" messages.

7. Resource Allocation

Many programs compete for resources. The OS allocates:

- CPU time
- Memory
- Disk space
- Printer access
- Network bandwidth

Analogy: Like a school timetable that allocates classrooms and teachers without conflicts.

8. Accounting

The OS records how system resources are used.

It keeps information about:

- CPU usage

- Memory usage
- Disk utilization
- User activity

Application: Cloud service providers use accounting information for billing customers based on resource usage.

9. Protection and Security

The OS protects the computer against unauthorized access and malware.

Security Features

- User Login
- Password Authentication
- File Permissions
- Encryption
- Firewall
- Antivirus Support

Example: Your smartphone asks for a PIN, fingerprint, or face recognition before allowing access.

Summary Table of OS Services

OS Service	Purpose	Everyday Example
User Interface	User interaction	Windows Desktop
Program Execution	Run applications	Opening Chrome

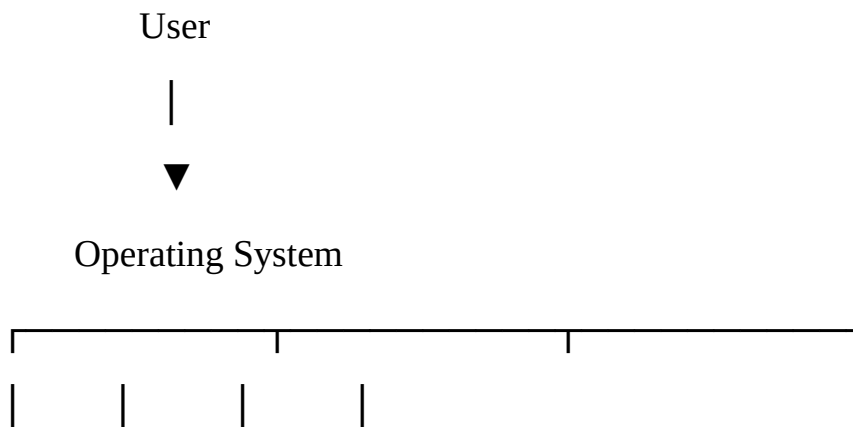
OS Service	Purpose	Everyday Example
I/O Operations	Manage devices	Printing documents
File Management	Organize files	Saving assignments
Communication	Exchange information	WhatsApp, Email
Error Detection	Detect faults	Disk Full warning
Resource Allocation	Manage CPU & Memory	Running multiple apps
Accounting	Monitor resource usage	Cloud billing
Protection & Security	Secure the system	Login password

Fun Fact

Did you know? Android, Windows, Linux, and macOS all provide the same basic operating system services. The difference lies in **how** they implement them and the user experience they offer.

Suggested Classroom Visuals

Diagram 1: Operating System Services



Program File Mgmt I/O Mgmt Security
Execution

Diagram 2: Program Execution Flow

User Clicks Program

|



Operating System

|

Load into RAM

|

Allocate CPU

|

Execute

|

Display Output

Diagram 3: File Management

User

|

Create / Copy / Rename / Delete

|

Operating System

|
Hard Disk / SSD

Flowchart: Requesting an OS Service

Application



Request OS Service



Operating System



Resource Allocation



Perform Task



Return Result

3. Real-World / Industry Applications (≈ 10 Minutes)

Operating system services are used everywhere:

- **Banking:** Secure transaction processing and file management.
- **Hospitals:** Real-time patient monitoring and medical record storage.
- **Cloud Computing:** Resource allocation, virtual machines, and user accounting.

- **Smartphones:** Camera access, storage management, touch input, and notifications.
- **Industrial Automation:** Communication between sensors, controllers, and machines.
- **Educational Institutions:** User authentication, shared storage, printers, and computer laboratories.

Industry Insight:

Companies such as Google, Amazon, Microsoft, and IBM rely on operating system services to manage millions of users, execute applications, protect data, and allocate computing resources efficiently in large-scale data centers.

4. Summary & Q&A ($\approx$ 5 Minutes)

Key Takeaways

- The Operating System provides essential services that make computers easy to use.
- Applications depend on OS services to execute programs, manage files, communicate, and use hardware.
- Major services include User Interface, Program Execution, I/O Operations, File Management, Communication, Error Detection, Resource Allocation, Accounting, and Protection & Security.
- Every modern computing device—from smartphones to cloud servers—relies on these services.

Quick Revision Questions

1. What are Operating System Services?
2. Explain the Program Execution service with an example.
3. Why is Resource Allocation important?
4. Differentiate between File Management and I/O Management.

5. How does the OS improve system security?

Typical Student Doubts

- Does every application directly access hardware?
 - Why do we need file permissions?
 - What happens if the OS cannot allocate memory?
 - How is CPU time shared among multiple applications?
 - What is the difference between security and protection?
-

Mentorship Note

Understanding **Operating System Services** is one of the most important foundations in Computer Science. Whether you become a **Software Developer, Cloud Engineer, System Administrator, Cybersecurity Analyst, DevOps Engineer, Network Engineer, or Embedded Systems Developer**, you will regularly interact with OS services through APIs, programming languages, and development tools. Mastering this topic will help you design efficient software, troubleshoot system issues, optimize performance, and build secure applications—skills that are highly valued in modern IT careers.

Lecture Content – Unit 1: Introduction to Operating System

Topic: Kernel, Shell and System Call

Course: Operating Systems (DI03016061)

Programme: Diploma Engineering – Information Technology

Duration: 60 Minutes

1. Hook / Introduction (≈ 5 Minutes)

Classroom Hook

Ask the students:

"When you click the Save button in MS Word, how does your file actually get stored on the hard disk? Does MS Word directly communicate with the hardware?"

Most students assume that the application directly interacts with the hardware. Explain that this is **not true**. Every request first goes through the **Operating System**.

Imagine a company:

- The **User** is the customer.
- The **Shell** is the receptionist who receives requests.
- The **Kernel** is the manager who makes decisions and allocates resources.
- The **Hardware** is the staff performing the actual work.

Whenever an application needs a service such as reading a file, printing a document, or accessing memory, it uses a **System Call** to request help from the Operating System.

Learning Outcomes

After completing this lecture, students will be able to:

- Define Kernel, Shell, and System Call.
- Explain the functions of the Kernel.
- Describe different types of Shells.
- Understand the role of System Calls.
- Illustrate the interaction between User, Shell, Kernel, and Hardware.

2. Core Concepts (≈ 40 Minutes)

Introduction

The Operating System is divided into several components. Among them, **Kernel**, **Shell**, and **System Calls** are the most important because they allow users and applications to communicate with computer hardware efficiently.

A. Kernel

Definition

The **Kernel** is the **core (heart)** of the Operating System. It loads into memory when the computer starts and remains active until the system shuts down.

The Kernel directly communicates with hardware and manages all system resources.

Functions of Kernel

- Process Management
- Memory Management
- CPU Scheduling
- Device Management
- File System Management
- Interrupt Handling
- Security and Protection

Example

When you open Google Chrome, the Kernel:

- Allocates memory.
- Schedules CPU time.
- Loads required files.

- Controls input/output devices.

Without the Kernel, no software can use the computer hardware.

Real-Life Analogy

Imagine a **school principal**.

Teachers, students, laboratories, classrooms, and staff work under the principal's supervision.

Similarly, the Kernel controls all computer resources.

Types of Kernel

1. Monolithic Kernel

All operating system services run inside the kernel.

Examples:

- Linux
- UNIX

Advantages

- Fast
- Efficient

Disadvantages

- Large size
 - Difficult to maintain
-

2. Microkernel

Only essential services remain inside the kernel.

Other services execute in user space.

Examples

- MINIX
- QNX

Advantages

- More secure
- Better reliability

Disadvantages

- Slightly slower
-

B. Shell

Definition

The **Shell** is the user interface of the Operating System.

It accepts commands from the user and forwards them to the Kernel.

The Shell acts as an interpreter between the user and the operating system.

Functions of Shell

- Accept user commands
 - Execute programs
 - Manage files and directories
 - Run scripts
 - Display results
-

Types of Shell

Command Line Shell (CLI)

Users type commands using a keyboard.

Examples

- Bash
- PowerShell
- CMD
- Zsh

Example Commands

dir

cd

mkdir

copy

Graphical Shell (GUI)

Users interact through icons, windows, menus, and buttons.

Examples

- Windows Desktop
- GNOME
- KDE
- macOS Finder

Real-Life Example

A restaurant waiter accepts your order and communicates it to the kitchen.

Similarly, the Shell receives commands from users and sends them to the Kernel.

C. System Call

Definition

A **System Call** is a programming interface that allows an application program to request services from the Operating System.

Applications cannot directly access hardware.

Instead, they request services using System Calls.

Working of System Call

Example:

Suppose MS Word wants to save a document.

The sequence is:

1. User clicks **Save**
 2. MS Word generates a **System Call**
 3. Kernel receives the request
 4. Kernel writes data to the hard disk
 5. Confirmation is returned to the application
-

Types of System Calls

Process Control

Examples

- Create Process
- End Process

- Load Program
-

File Management

Examples

- Open File
 - Read File
 - Write File
 - Close File
-

Device Management

Examples

- Read Keyboard
 - Write Printer
 - Access USB Device
-

Information Maintenance

Examples

- Get Time
 - Get Date
 - Get System Information
-

Communication

Examples

- Send Message
- Receive Message
- Network Communication

Fun Fact

Every time you unlock your smartphone, play a game, print a document, or upload a file, hundreds of **system calls** are executed in the background within a fraction of a second. Users never notice them, but they are essential for every operation.

Comparison: Kernel vs Shell vs System Call

Feature	Kernel	Shell	System Call
Meaning	Core of Operating System	User Interface	Request to OS
Directly Accesses Hardware	Yes	No	No
User Interaction	No	Yes	No
Main Function	Manage Resources	Accept Commands	Request OS Services
Example	Linux Kernel	Bash, CMD	open(), read(), write()

Suggested Classroom Visuals

Diagram 1: Operating System Architecture

+-----+

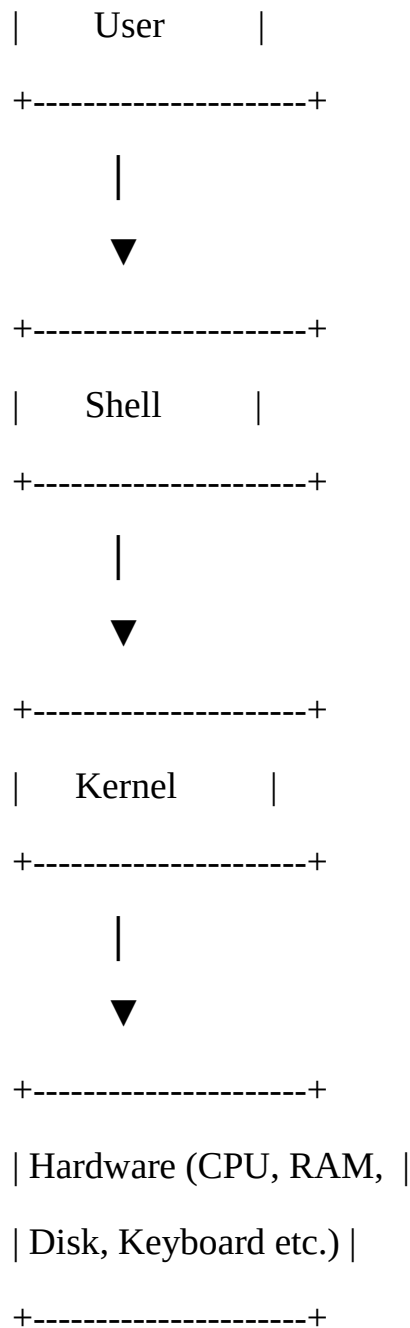


Diagram 2: System Call Flow

User



Application



System Call



Kernel

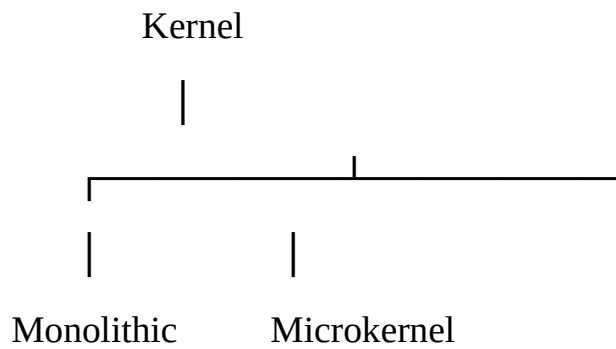


Hardware



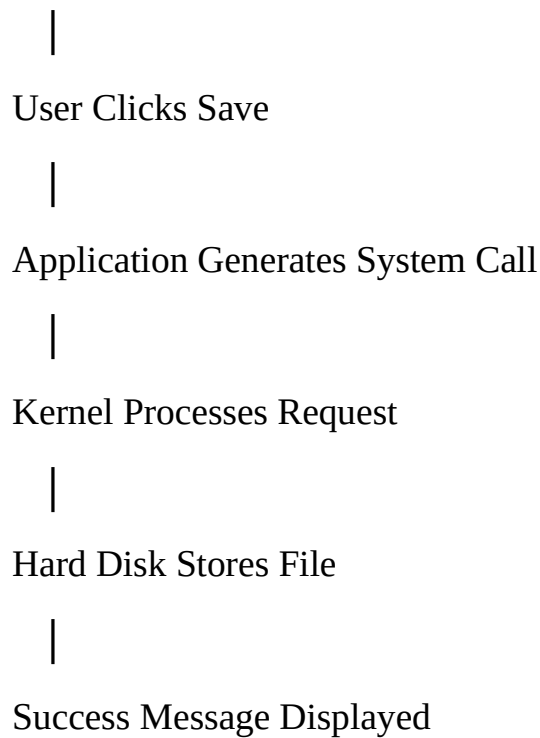
Output

Diagram 3: Types of Kernel



Flowchart: File Save Operation

Start



3. Real-World / Industry Applications ($\approx$ 10 Minutes)

- **Linux Servers:** The Linux Kernel manages cloud servers used by Google, Amazon, and Microsoft.
- **Android Smartphones:** Android uses the Linux Kernel for memory management, process scheduling, and device control.
- **Banking Systems:** Every ATM transaction uses system calls to access files, verify users, and communicate securely with bank servers.
- **Automotive Industry:** Modern vehicles use kernels in embedded operating systems to manage sensors, braking systems, and infotainment.
- **Software Development:** Developers use Bash, PowerShell, or Terminal shells to compile code, automate tasks, and deploy applications.

Industry Insight:

Cloud platforms such as AWS, Microsoft Azure, and Google Cloud rely heavily on

Linux kernels and shell scripting for automation, server management, and large-scale infrastructure operations.

4. Summary & Q&A ($\approx$ 5 Minutes)

Key Takeaways

- The **Kernel** is the heart of the Operating System and manages hardware resources.
- The **Shell** provides an interface for users to interact with the Operating System.
- A **System Call** is the mechanism through which application programs request services from the Kernel.
- Applications never communicate directly with hardware; they always use Operating System services.

Quick Revision Questions

1. What is a Kernel?
2. Why is the Kernel called the heart of the Operating System?
3. Explain the role of the Shell with an example.
4. What is a System Call?
5. Differentiate between Kernel, Shell, and System Call.
6. Name two types of Kernel.
7. Give examples of file-related system calls.

Typical Student Doubts

- Can an application directly access hardware?
- Is GUI also a type of Shell?

- Why are system calls required?
 - What is the difference between a Kernel and an Operating System?
 - Which Kernel does Linux use?
-

Mentorship Note

Kernel, Shell, and System Calls form the backbone of modern operating systems. A clear understanding of these concepts is essential for advanced subjects such as **Linux Administration, System Programming, Operating System Design, Embedded Systems, Cloud Computing, Cybersecurity, DevOps, and Software Engineering**. During internships and technical interviews, you may be asked to explain how an application interacts with hardware or why Linux is preferred for servers. Mastering these topics will strengthen your programming skills, improve debugging abilities, and prepare you for careers as a **System Administrator, Linux Engineer, DevOps Engineer, Cloud Engineer, Embedded Systems Developer, or Cybersecurity Professional**.